ENGAGEMENT REPORT

Trendy Golf — Shopify Theme Build

Claude Code engagement summary · eComplete delivery

19 Mar 2026 → 18 Apr 2026 · generated 18 Apr 2026

906	22h 7m	24h 6m	115	708.6M
PROMPTS	CLAUDE RUNTIME	DIRECTION TIME	COMMITS	TOKENS PROCESSED
	parallelisable with other projects	hands-on reading & prompting	across 11 active days	

How these numbers are calculated. Every Claude Code session writes a timestamped event log to disk. The `build_dashboard.py` script walks those logs event-by-event and measures the gap between each pair of consecutive events.

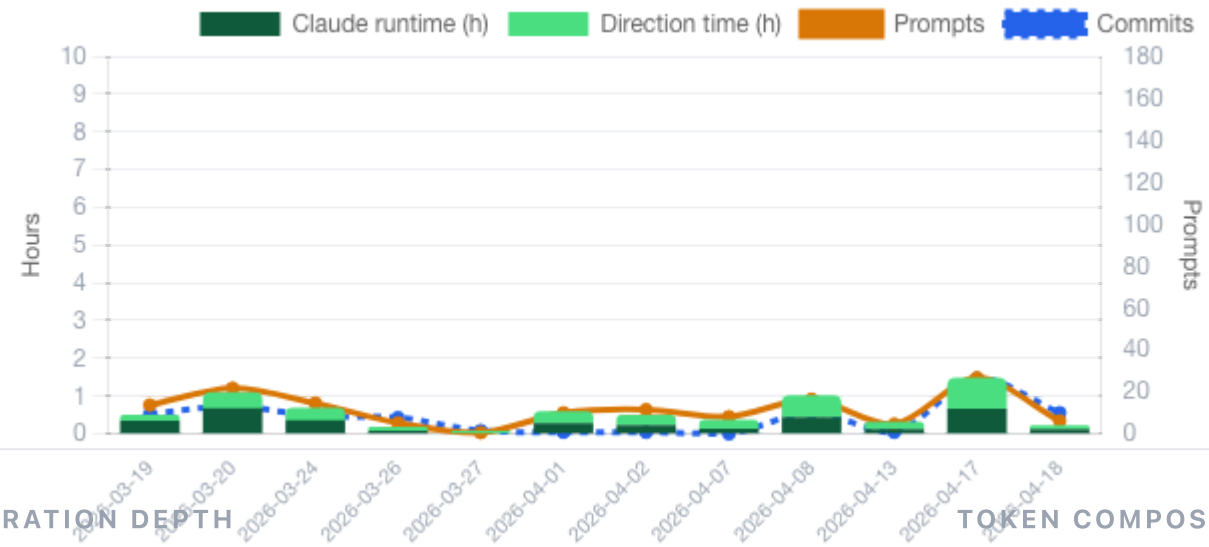
Claude runtime (22h 7m) is the sum of gaps that end in an assistant message or tool result — i.e. Claude itself was computing to produce that event. Capped at 15 minutes, since Claude rarely pauses longer mid-chain. This work ran in parallel with other projects.

Direction time (24h 6m) is the sum of gaps that end in a user prompt — reading output, reviewing diffs, and writing the next instruction. Capped at 30 minutes (gaps longer than that are treated as genuine breaks: lunch, meetings, next day).

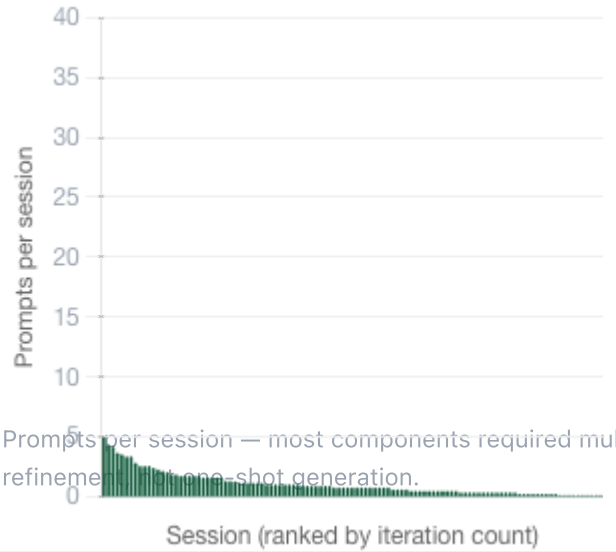
TOKEN SANITY CHECK ON CLAUDE RUNTIME

Independently: Opus generated **3.8M output tokens** at ~60 tokens/sec ≈ **17.8h** of pure generation. Adding 5,124 tool calls (Bash, file I/O, subagents) lands in the 20-25h range — the log-derived **22h 7m** matches, which is a good cross-check on the methodology.

ACTIVITY OVER TIME



ITERATION DEPTH



Prompts per session — most components required multiple rounds of refinement, not one-shot generation.

TOKEN COMPOSITION

Cache reads dominate — repeated reads of repo context as work iterates on each component.

MODELS USED

MODEL	MESSAGES	INPUT	OUTPUT	CACHE READ	CACHE WRITE	TOTAL
`claude-opus-4-6`	5,858	340.5k	1.3M	351.1M	13.9M	**366.6M**
`claude-opus-4-7`	2,997	7.1k	2.5M	329.9M	9.5M	**341.9M**

BUILD PROCESS

PHASE 0

Scope & Spec

Audit references (scraped HTML first, repo source second). Write a component spec doc. Flag client questions and deferred scope.

`docs/build-playbook.md`

→

PHASE 1

Three-Round Audit

Parallel agents compare spec against scraped markup, Tailwind classes, and JS behaviour. No building until the spec survives three audits.

`/audit skill`

→

PHASE 2

Build

Liquid, Tailwind, and vanilla JS written to match scraped HTML exactly. Repo conventions enforced via memory and skill guidance.

`/build skill`

→

PHASE 3

Validate & Handover

Shopify theme validator, staging store check, visual diff vs. scraped baseline. Handover doc generated for continuity.

`/validate · /handover`

ENGAGEMENT INSIGHTS

Patterns observed from the session logs — what worked, where friction showed up, and what to invest in next.

Source: the `/insights` skill in Claude Code, which audits session logs and extracts wins, friction patterns, and suggestions. The raw report lives at `~/.claude/usage-data/report.html`; the Shopify-relevant findings have been extracted and summarised below.

Fully achieved (62) · Mostly achieved (49) · Partially (9) · Not achieved (2)

What's working

REPEATABLE PIPELINE

Spec-driven section builds

The repo's scraped HTML + Figma references + per-component spec docs + three-round audit loop is a genuinely repeatable system, not ad-hoc prompting. Header, hero, footer, predictive search, collection pages, carousels — all shipped through the same workflow.

DOCUMENTATION AS HANDOVER

Docs ship alongside code

22 spec docs and handover markdown treat documentation as a first-class deliverable — decisions, deferred items, and client questions are captured in-repo. The engagement crossed 111 sessions without losing context, because the docs carried it.

TIGHT COMMIT CADENCE

115 commits, mostly green

Small, frequent commits with the Shopify GitHub integration auto-syncing to staging meant changes could be reviewed in the browser within minutes of landing. 111 of 122 tracked outcomes fully or mostly achieved (~91%).

STRENGTH AREAS

Multi-file refactors & debugging

Claude's top capabilities in this project: multi-file changes (52 sessions), debugging (25), explanations (23), and correct code edits (15) — matches a theme build where consistency across Liquid/CSS/JS is critical.

Where friction showed up

DESIGN FIDELITY

Guessing visual details instead of verifying

The single biggest friction source: Claude fabricating arrow SVGs, progress bar colours, scrollbar styling, or border radii instead of checking the scraped HTML or Figma first. Required extra correction rounds on the featured collection swiper, filter sidebar, and collection pages.

BUILD PIPELINE

Wrong-file edits & rebuild wipeouts

Claude edited compiled `theme.css` instead of the Tailwind `input.css` in one session; a Tailwind rebuild during footer work wiped uncompiled CSS and needed an emergency hotfix. Both preventable with hooks or CLAUDE.md pinning.

SCOPE DISCIPLINE

Jumping ahead of the spec

On a few sessions Claude started implementing before the spec doc was locked — leading to rework when the actual request was to update the doc first. The "scope-first" pattern works when stated explicitly at session start.

ROOT CAUSE HUNTS

Investigating wrong hypotheses first

The Splide crash on product pages and the nested style-tags bug both saw Claude chase the wrong root cause before landing on the real fix. Faster resolution when asked upfront to enumerate 2-3 hypotheses before picking one.

Suggested next moves

AUTOMATION

Codify the section-build flow as a Skill

Every section followed the same shape: check scraped HTML → check Figma → write Liquid/Tailwind → validate → commit. Wrap that into a `/section` skill that asks for the Figma URL upfront, forces the verification step, and refuses to touch compiled `theme.css`.

SAFETY

Add PostToolUse hooks

A PostToolUse hook that (a) blocks writes to `assets/theme.css`, (b) runs `shopify theme check` after any Liquid edit, and (c) validates section JSON would have caught both the CSS wipeout and the stray-brace incidents before they reached main.

STRETCH GOAL

Figma-to-Shopify pixel-match loop

Combine the Figma MCP with a Playwright MCP: Claude builds the section, screenshots the staging store, diffs against the Figma frame, and self-corrects up to N rounds. Turns the multi-round refinement cycles we saw in this engagement into hands-off builds.

GOVERNANCE

Extend the spec library across clients

The 22-doc library here (build-playbook, foundation, header, footer, product-page, etc.) is a template other eComplete theme builds could inherit. Publishing it as an internal starter repo would shortcut the Phase 0 scope work for the next engagement.

SKILLS USED

🔍

/audit

Triple-round reference audit

Runs three parallel agents against scraped HTML, repo source, and the spec doc. Flags markup, style, or behaviour inconsistencies before a single line of theme code is written.

🏗️

/build

Spec-driven component build

Reads the component spec, scraped HTML, and reference React/TSX. Writes Liquid, Tailwind classes, and vanilla JS in the exact style of the repo — no framework drift, no invented settings.

✅

/validate

Theme + browser validation

Runs Shopify's theme validator via MCP, checks schema correctness, then drives the staging storefront to verify the feature renders and behaves against the scraped baseline.

📦

/handover

Session handover generator

Scans the session log and diff, emits a handover markdown: files touched, decisions made, deferred items, and links into the spec docs for continuity across builds.

TOOLS & INTEGRATIONS

MCP (Model Context Protocol) is the standard for wiring Claude directly into external systems. The Shopify Dev MCP was the workhorse of this engagement — every piece of Liquid, every section schema, and every GraphQL query was validated against Shopify's authoritative docs before shipping.

Shopify Dev MCP

147 tool calls this engagement

Official Shopify MCP — theme validation, Liquid docs, GraphQL schema introspection, component validation.

`validate_theme`	76	`learn_shopify_api`	33	`search_docs_chunks`	31
`introspect_graphql_schema`	4	`fetch_full_docs`	3		

APIs & HTTP Integrations

`Shopify Admin GraphQL`	menus, themes, products · 2024-10 & 2025-01
`Shopify Storefront / CDN`	staged theme push & asset delivery
`jsDelivr CDN`	Splide.js carousel assets
`Sanity CMS`	source-site magazine content

Scraping Pipeline

`scrape.mjs`	main page HTML scraper
`scrape-assets.mjs`	CSS / JS / image asset pull
`scrape-remaining.mjs`	follow-up pass for missed routes
`extract-stream-data.mjs`	React Router loader JSON extraction
`download-js.sh`	bulk JS download helper

Claude Code Native Tools

`Bash`	1,169 calls
`Read`	1,255 calls
`Edit`	1,004 calls
`Write`	202 calls
`Grep`	591 calls
`Glob`	178 calls
`Agent (subagents)`	193 calls
`WebFetch / WebSearch`	24 calls

Build & Shell Utilities

`Tailwind CSS CLI`	compile theme.css from tokens
`Splide.js`	carousel library (matching source)
`curl`	Shopify API + CDN asset fetches
`jq`	JSON inspection of scraped stream data
`perl`	regex transforms on scraped HTML/CSS
`git + gh`	Shopify GitHub integration auto-syncs theme

DOCUMENTATION LIBRARY (23 SPECS WRITTEN)

Product Page	1126 lines	docs/product-page.md
Sections	902 lines	docs/sections.…
Cart Drawer	535 lines	docs/cart-drawer.md
Rewards	498 lines	docs/rewards.md
Article Template	489 lines	docs/article-template.md
Blog Template	359 lines	docs/blog-template.md
Build	346 lines	docs/BUILD.md
Shoe Finder	332 lines	docs/shoe-finder.md
Accounts	328 lines	docs/accounts.…
Handover 2B Collection	315 lines	HANDOVER-2B-COLLECTION.md
Article Migration	293 lines	docs/article-migration.md
Brand Listing	282 lines	docs/design/bran…listing.md
Collection	273 lines	docs/collection…
Launches	251 lines	docs/launches.…
Temp Magazine Articles	246 lines	docs/temp-magazine-articles.md
Build Playbook	219 lines	docs/build-playbook.md
Project	214 lines	docs/PROJECT.md
Help Centre	130 lines	docs/help-centre.md
Header	110 lines	docs/header.md
Client Questions	85 lines	docs/client-questions.md
Foundation	55 lines	docs/foundation…
Product Card	51 lines	docs/product-card.md
Footer	22 lines	docs/footer.md